



MEMOIRE

présenté à

L' École Supérieure des Sciences Appliquées et de Management

en vue de l'obtention du

**Diplôme National d'Ingénieur en :
Informatique**

**Option :
Cloud-cybersécurité**

par

Dhia Eddine Mokhtar

**Développement d'un générateur de site et mise en place d'un
ERP de gestion des garderies scolaire .**



DEDICATION

À ma très chère mère, autant de phrases aussi expressives sient-elles ne sauraient exprimer mn respect, mn amour éternel et ma considération pour les sacrifices que vous avez consenti pur mon éducation et mn bien-être. Vtre affection inconditionnée me couvre, votre sagesse me guide et vtre présence à mes côtés a toujours été ma source de force pur affronter les différents obstacles. Je vous remercie pur tout le soutien et l'amour que vus portez à mon égard et j'espère que votre bénédiction m'accompagne toujours. Que ce modeste travail sit l'exaucement de vs vœux tant frmulés, le fruit de vos innombrables sacrifices.

Puisse Dieu, le Très Haut, vus accrder santé, bonheur et longue vie.

À mon cher père, qui n'a jamais dit nn à mes exigences et n'a épargné aucun effrt pour me rendre heureuse.

À ma chère sœur, en souvenir d'une enfance dnt nus avns partagé les meilleurs et les plus agréables mments. Tu m'as chaleureusement sutenu, récnforté et encouragé tout au lng de mon parcur.

Puissent ns liens fraternels se conslider et se pérenniser encre plus.

À mon cher frère, à qui je dois mon sourire.

À tute ma famille, à tous mes ami(e)s, Je vus dédie ce travail en recnnaissance de l'amour que vous m'ffrez quotidiennement et votre bnté exceptionnelle. Que Dieu, le Tout Puissant, vous garde et vous procure santé et bonheur

À tus ceux que j'aime et ceux qui m'aimaient.



REMERCIEMENT

Au terme de ce travail, je voudrais remercier tous ceux qui, sans leur aide inestimable, ce projet n'aurait jamais été mené à son terme.

Mes remerciements s'adressent particulièrement à : Sami Harrabi, Responsable de d'Horizon Data, pour m'avoir prodigué l'honneur de travailler dans son équipe.

Mr. amine harrabi et Mr. Bahri Yassine, mes encadrants à Horizon Data, à qui je tiens à exprimer toute ma gratitude pour l'aide qu'ils m'ont apportée durant toutes les phases de ce stage. Leur disponibilité, leur encadrement, leur pédagogie et leurs conseils m'ont été précieux pour atteindre les objectifs de ce projet dans les délais convenus.

Mme. amal.bouaziz, mon encadrante à SESAME, pour la qualité de son enseignement, son encouragement et ses conseils qui ont été très bénéfiques pour le bon cheminement de ce rapport.

Tous les enseignants qui ont participé à mon évolution scientifique durant les années écoulées de ma formation à SESAME.

Enfin, j'adresse mes plus vifs remerciements aux membres du jury pour m'avoir honoré en acceptant d'examiner ce travail.

Sans oublier tous ceux qui ont participé de près ou de loin à l'accomplissement de ce projet. Il n'est malheureusement pas possible de les citer toutes ici, mais elles se reconnaîtront.



TABLE DES MATIÈRES

LISTE DES FIGURES	vi
LISTE DES ABRÉVIATIONS	vii
INTRODUCTION GÉNÉRALE	viii
1 Étude Préalable	2
1.1 INTRODUCTION	3
1.2 Carte générale du travail	3
1.2.1 Présentation de L'organisme d'accueil	3
1.2.1.1 Services	4
1.2.2 Contexte du projet	4
1.3 Étude de l'existant	5
1.3.1 Systèmes traditionnels de gestion des garderies	6
1.3.2 Solutions informatiques existantes	6
1.3.2.1 Étude Comparatif	7
1.4 Solution proposées	8
1.4.1 Architecture microservices et microfrontend	8
1.4.1.1 Synthèse pour le choix de la solution	8
1.4.2 Gestion centralisée des informations	9
1.4.3 Communication fluide entre les parties prenantes	9
1.4.4 Intégration d'un module d'intelligence artificielle	10
1.4.5 Sécurité et protection des données	10
1.4.6 Déploiement avec Kubernetes	11
1.5 Méthodologie adoptée	11
1.5.1 Présentation de SCRUM	12
1.5.2 Formalisme adopté	12
1.6 CONCLUSION	13

2	Analyse et spécification des besoins	14
2.1	INTRODUCTION	15
2.1.1	Analyse des besoins	15
2.1.1.1	Analyse des besoins fonctionnels	15
2.1.1.2	Analyse des besoins non fonctionnels	16
2.2	Spécification des besoins	17
2.2.1	Diagramme de cas d'utilisation Global	17
2.3	Diagramme de classes	19
2.4	Planification de travail	20
2.4.1	Répartition des releases	20
2.4.2	Product backlog globale	22
2.5	Architecture de l'application	22
2.5.1	Architecture logique	22
2.5.1.1	Architecture logique frontEnd	24
2.5.1.2	Architecture logique de Django	25
2.5.2	Architecture physique	27
2.5.3	Architecture microservices	28
2.6	Environnement de travail	29
2.6.1	Environnements de développement matériel	29
2.6.2	Environnements de développement logiciel	30
2.7	CONCLUSION	30
3	Release	32
3.1	INTRODUCTION	33
3.2	Sprint 1 : Authentification et gestion des classes	33
3.2.1	Objectifs du sprint 1	33
3.2.2	Analyse du sprint 1	34
3.2.3	Diagramme de cas d'utilisation	35
3.2.4	Description textuelle des cas d'utilisation	36
3.2.4.1	Description textuelle du cas d'utilisation « S'authentifier »	36
3.2.4.2	Description textuelle du cas d'utilisation «Créer une classe»	37
3.2.4.3	Description textuelle du cas d'utilisation «Associer un enseignant à une classe»	38
3.2.5	Diagramme de classes	39
3.2.6	Réalisation	40
3.2.6.1	Technologies utilisées	40

CONCLUSION GÉNÉRALE	43
BIBLIOGRAPHIE	44



LISTE DES FIGURES

1.1	Logo de l'entreprise	4
1.2	Pourcentages des méthodes agiles utilisées [1]	12
2.1	Diagramme de cas d'utilisation global	17
2.2	Diagramme de classe global	19
2.3	Architecture logique de l'application [2]	23
2.4	Architecture logique Front-End (Angular) [3]	25
2.5	Architecture logique du backend (Django) [4]	26
2.6	Architecture physique globale [5]	27
2.7	Architecture monolithique versus architecture microservices [6]	29
3.1	Diagramme de cas d'utilisation	35
3.2	Diagramme de classes	39
3.3	Interface d'authentification	40
3.4	Interface d'authentification	41
3.5	modification d'un enseignant	41
3.6	Interface enseignant : liste des élèves de sa classe	42



LISTE DES ABRÉVIATIONS

AAA Aaa Aaa Aaa

BB Bbb Bbb

CCC Ccc Ccc Ccc



INTRODUCTION GÉNÉRALE

Le domaine de l'éducation connaît une transformation numérique croissante, poussée par l'évolution des technologies, des attentes des parents et des besoins d'organisation des établissements scolaires. Dans ce contexte, les écoles, collèges et autres structures éducatives doivent adopter des solutions informatiques efficaces pour gérer les interactions entre les différents acteurs : administrateurs, enseignants, élèves et parents.

Cependant, la gestion manuelle ou partiellement informatisée des classes, des cours, du suivi pédagogique et de la communication entre les parties prenantes engendre souvent des lenteurs, des redondances et une faible traçabilité de l'information. Pour pallier ces difficultés, il devient essentiel de mettre en place un système d'information centralisé, flexible et évolutif.

C'est dans cette optique que s'inscrit notre projet de fin d'études, qui consiste à concevoir et développer une application web éducative. Ce système vise à faciliter la gestion des classes, le suivi des élèves, la distribution de contenus pédagogiques et la communication avec les parents. Il propose une interface adaptée à chaque profil utilisateur et permet une administration fluide des données éducatives.

Le développement de cette application a été mené en adoptant une approche agile basée sur la méthode Scrum. Le projet repose sur une architecture moderne intégrant Angular pour le front-end et Django REST Framework pour le back-end, avec une organisation orientée microservices. Une attention particulière a été accordée à la sécurité, à l'évolutivité, et à la possibilité d'intégrer ultérieurement des modules d'intelligence artificielle.

Le présent rapport retrace les différentes phases de réalisation de ce projet. Il est structuré en cinq chapitres principaux :

Le premier chapitre introduit le contexte général du projet, présente le besoin auquel répond l'application et décrit la méthodologie choisie. Le deuxième chapitre détaille l'analyse des besoins fonctionnels et non fonctionnels, le découpage en sprints, l'architecture adoptée et les outils utilisés.

Les troisième, quatrième et cinquième chapitres couvrent les phases de développement successives correspondant aux différents sprints Scrum, en mettant en avant les fonctionnalités réalisées, les interfaces implémentées, et les résultats obtenus. Le rapport se conclut par une synthèse du travail accompli, les difficultés rencontrées et les perspectives d'amélioration futures.

Étude Préalable

Sommaire

1.1	INTRODUCTION	3
1.2	Carte générale du travail	3
1.2.1	Présentation de L'organisme d'accueil	3
1.2.2	Contexte du projet	4
1.3	Étude de l'existant	5
1.3.1	Systèmes traditionnels de gestion des garderies	6
1.3.2	Solutions informatiques existantes	6
1.4	Solution proposées	8
1.4.1	Architecture microservices et microfrontend	8
1.4.2	Gestion centralisée des informations	9
1.4.3	Communication fluide entre les parties prenantes	9
1.4.4	Intégration d'un module d'intelligence artificielle	10
1.4.5	Sécurité et protection des données	10
1.4.6	Déploiement avec Kubernetes	11
1.5	Méthodologie adoptée	11
1.5.1	Présentation de SCRUM	12
1.5.2	Formalisme adopté	12
1.6	CONCLUSION	13

1.1 INTRODUCTION

Dans un environnement où la transformation numérique impacte tous les secteurs, le secteur de l'éducation et de la petite enfance n'échappe pas à cette dynamique. Les établissements tels que les crèches, bien qu'ils jouent un rôle essentiel dans le développement des jeunes enfants, souffrent encore d'un manque d'outils numériques adaptés à leur gestion quotidienne. Les besoins sont variés, allant du suivi des enfants à la communication avec les parents, en passant par la planification des activités pédagogiques et la gestion administrative, souvent traités de manière manuelle ou éparpillée. C'est dans ce contexte qu'a été initié le projet de création d'une application web CRM spécifiquement conçue pour la gestion d'une crèche. Cette application a pour but de centraliser et d'automatiser l'ensemble des processus liés à la gestion pédagogique et administrative, tout en facilitant une communication fluide entre les différents intervenants : enseignants, parents, élèves et administration. L'objectif principal de cette application est de proposer une plateforme intuitive et sécurisée qui permettra aux enseignants de gérer aisément leurs cours et classes, aux élèves d'accéder aux ressources pédagogiques, aux parents de suivre en temps réel les activités et le développement de leurs enfants, et à l'administration de superviser l'ensemble des opérations, y compris les documents officiels et les affectations. Ce projet constitue ainsi une réponse concrète à un besoin réel d'organisation, de traçabilité et de modernisation des structures éducatives, s'inscrivant dans une démarche de digitalisation intelligente et humaine au service de la petite enfance. Dans ce rapport, je présenterai d'abord le cadre de ce stage, puis je détaillerai les différentes étapes de la conception de l'application, ses fonctionnalités, les technologies employées, ainsi que les compétences techniques et personnelles que j'ai acquises.

1.2 Carte générale du travail

1.2.1 Présentation de L'organisme d'accueil

Horizon Data est une société de services informatiques spécialisée dans l'étude, le conseil et le développement informatique.

Crée en 2013, la société a acquis un savoir-faire, des compétences et de l'expérience lui permettant de mener à bien tout projet informatique avec tous ses aspects (spécification, conception, développement, test, validation, déploiement, assistance et maintenance).

Le siège social de cette société est situé à Technopark Elgazela Ariana-Tunis.

La figure 1.1 présente le logo de notre société d'accueil.



FIGURE 1.1 – Logo de l'entreprise

1.2.1.1 Services

Horizon Data est spécialisée dans trois activités principales à savoir :

***Le réseau** : C'est à dire l'administration réseau et sécurité réseau.

***La maintenance informatique** : Ce qui concerne le backup, les sauvegardes et les antivirus.

***Le développement** : -web.

-mobile.

1.2.2 Contexte du projet

La gestion des établissements éducatifs, notamment ceux consacrés à la petite enfance tels que les garderies, représente une tâche à la fois complexe et exigeante. Ces structures doivent non seulement encadrer les enfants, mais aussi organiser des activités pédagogiques, assurer

un suivi individualisé, communiquer efficacement avec les parents et gérer une multitude de documents administratifs. Dans de nombreuses garderies, ces fonctions sont encore exécutées de manière traditionnelle, souvent sur papier ou à l'aide de solutions informatiques rudimentaires et non intégrées, ce qui entraîne une perte de temps, une surcharge de travail pour le personnel, ainsi que des erreurs ou des oublis dans le traitement des informations. Avec l'évolution rapide des technologies de l'information et l'essor des plateformes numériques dans tous les secteurs, le domaine de la petite enfance ne peut se permettre de rester à l'écart de cette transformation. Il est désormais crucial d'équiper ces structures d'outils numériques adaptés à leurs besoins spécifiques, capables de centraliser l'information, d'améliorer la communication et de faciliter la prise de décision. C'est dans cette perspective que s'inscrit le projet réalisé dans le cadre de mon stage chez Horizon Data. Ce projet consiste en la conception et le développement d'une application web CRM dédiée à la gestion intégrale d'une garderie. Cette solution vise à moderniser et à numériser les divers processus internes d'un établissement de ce type, en intégrant des fonctionnalités pédagogiques, administratives et relationnelles. Le choix de développer un CRM (Customer Relationship Management) dans un contexte éducatif repose sur l'idée que les enfants, les parents et les enseignants forment un réseau d'interactions qu'il est nécessaire d'organiser et de renforcer grâce à la technologie. Ainsi, ce projet répond à un besoin réel d'organisation, de transparence et d'efficacité dans la gestion quotidienne.

1.3 Étude de l'existant

Avant de commencer le développement de la nouvelle solution, il est crucial d'examiner les outils et systèmes déjà en place dans le secteur de la gestion des garderies. Cette analyse des pratiques actuelles permet d'identifier les limites des solutions existantes ainsi que les besoins spécifiques qui ne sont pas satisfaits c'est pour ça on a choisi de travailler sur deux applications existantes dans le marché français.

1.3.1 Systèmes traditionnels de gestion des garderies

Dans de nombreuses garderies, la gestion est encore réalisée de manière manuelle ou à l'aide de logiciels de gestion qui ne répondent pas aux exigences particulières du secteur. Cela inclut souvent l'utilisation de tableurs, de documents papier ou de logiciels génériques qui ne tiennent pas compte des spécificités de l'éducation préscolaire. Par exemple, les enseignants et les administrateurs utilisent fréquemment des carnets de suivi pour chaque enfant, où sont consignées les informations relatives aux absences, aux performances et aux activités pédagogiques. Cette méthode est non seulement chronophage, mais également sujette à des erreurs humaines. Certains établissements optent également pour des logiciels de gestion scolaire déjà disponibles, mais ceux-ci sont généralement conçus pour des écoles primaires ou secondaires et ne répondent pas toujours aux besoins spécifiques des garderies. Ces outils manquent souvent de la flexibilité nécessaire pour gérer les interactions particulières entre enseignants, élèves et parents, ainsi que de fonctionnalités adaptées à la gestion de la petite enfance, telles que le suivi individualisé des enfants ou la planification d'activités pédagogiques simples.

1.3.2 Solutions informatiques existantes

Il existe de nombreuses solutions logicielles sur le marché pour la gestion des établissements éducatifs, mais peu d'entre elles sont spécifiquement adaptées aux besoins des garderies. Certaines entreprises proposent des logiciels de gestion pour les crèches, mais ces derniers se révèlent souvent limités en termes de fonctionnalités, manquent de personnalisation ou ne sont pas suffisamment intuitifs pour les utilisateurs non technophiles, tels que les parents et les enseignants. De plus, la majorité de ces solutions n'assurent pas une communication fluide entre les différents intervenants, ni un suivi global des enfants sur des aspects variés tels que les performances académiques, les activités extrascolaires ou les interactions avec les parents. Les plateformes actuellement disponibles dans le secteur éducatif sont souvent monolithiques, intégrant à la fois le frontend et le backend dans une seule structure. Bien que ce type de solution puisse offrir des fonctionnalités intéressantes, il manque de la flexibilité nécessaire pour s'adapter aux besoins évolutifs des garderies. La gestion des utilisateurs, l'intégration de nouvelles fonctionnalités ou

l'ajout de modules complémentaires deviennent rapidement des tâches complexes à maintenir et à faire évoluer.

1.3.2.1 Étude Comparatif

Critère	Gestion traditionnelle	Application web de gestion des garderies
Méthode de saisie des données	Papier, feuilles Excel, carnets	Interface en ligne, formulaires numériques
Accessibilité des données	Sur place uniquement, souvent dans des classeurs	Accessible à distance depuis n'importe quel appareil connecté
Sécurité	Risque de perte, d'erreur ou de vol de données	Sauvegardes automatiques, accès sécurisé avec authentification
Mise à jour des informations	Manuelle, lente, sujette à l'oubli	Instantanée et synchronisée pour tous les utilisateurs
Communication avec les parents	Téléphone, papier à remettre	Notifications automatiques, messagerie intégrée
Suivi des présences et activités	Feuilles manuelles, pointage papier	Système automatisé de pointage, tableau de bord en temps réel
Archivage	Archivage physique, encombrant	Archivage numérique, consultation rapide
Coût à long terme	Moindre au départ, mais coûteux en temps et en erreurs	Investissement initial, mais gain de temps et fiabilité accrue
Statistiques et rapports	Calculs manuels, tableurs compliqués	Génération automatique de rapports et graphiques
Écologie	Papier et encre utilisés en grande quantité	Dématérialisation totale ou partielle

TABLE 1.1 – Comparatif entre la gestion traditionnelle et une application web pour les garderies

1.4 Solution proposées

Le développement d'une solution CRM pour la gestion des garderies représente une réponse directe aux limitations des systèmes existants. Cette application web, qui sera conçue à l'aide de technologies modernes (Angular pour le frontend et Django pour le backend), répondra aux besoins spécifiques des garderies en matière de gestion des élèves, des enseignants, des parents, et de l'administration.

1.4.1 Architecture microservices et microfrontend

L'une des principales caractéristiques de cette solution est son architecture basée sur des microservices et des microfrontends. Cette approche permet de garantir plusieurs avantages clés.

1.4.1.1 Synthèse pour le choix de la solution

Scalabilité : L'architecture microservices permet à l'application de s'adapter facilement aux besoins croissants de l'établissement, tout en permettant l'ajout rapide de nouvelles fonctionnalités. Par exemple, l'ajout de nouveaux modules (suivi des repas, gestion des absences, etc.) pourra se faire indépendamment des autres parties du système.

Flexibilité : Chaque microservice est indépendant et peut être développé, testé, et déployé séparément, ce qui rend le processus de développement et de maintenance beaucoup plus agile.

Résilience : En cas de panne d'un microservice, l'application continue de fonctionner de manière partielle, ce qui améliore la disponibilité globale du système.

Séparation des responsabilités : L'utilisation de microfrontends permettra de séparer les différentes parties de l'interface utilisateur en composants indépendants, ce qui offrira une meilleure gestion du code frontend, tout en améliorant la performance et la maintenabilité.

Cette architecture découplée garantit que l'application pourra évoluer en fonction des besoins des utilisateurs et intégrer facilement des améliorations futures, telles que des modules d'intelligence artificielle ou des fonctionnalités avancées de gestion.

1.4.2 Gestion centralisée des informations

La solution CRM propose une plateforme centralisée où toutes les données relatives aux élèves, enseignants et parents sont stockées et mises à jour en temps réel. Chaque acteur (enseignant, parent, administrateur) dispose d'un compte personnalisé avec des accès et des privilèges définis, garantissant la confidentialité des informations.

Les parents peuvent suivre l'évolution de leur enfant grâce à un espace dédié, où ils peuvent consulter les activités, les évaluations, et les rapports de progrès. De leur côté, les enseignants auront accès à une vue complète des élèves de leurs classes, avec des outils pour ajouter des cours, des évaluations et des événements pédagogiques.

Le système centralisé offre plusieurs avantages :

Accessibilité : Tous les utilisateurs peuvent consulter les informations dont ils ont besoin en temps réel, que ce soit via une interface web ou mobile.

Gestion simplifiée : L'intégration des données au sein d'une même plateforme élimine les doublons et assure une gestion fluide et cohérente des informations.

Sécurité et confidentialité : L'application adopte des mesures de sécurité robustes, assurant la protection des données sensibles, avec des protocoles de chiffrement et des méthodes d'authentification sécurisées.

1.4.3 Communication fluide entre les parties prenantes

La solution CRM met un point d'honneur à améliorer la communication entre les enseignants, les parents, et l'administration. À travers l'application, les parents peuvent :

Recevoir des notifications instantanées : sur les événements importants concernant leurs enfants (activités, absences, notes).

Interagir directement avec les enseignants : via une messagerie sécurisée, facilitant ainsi la résolution rapide de toute question ou inquiétude.

Accéder à des rapports détaillés : sur les progrès et les activités de leurs enfants, avec une mise à jour régulière des informations.

De leur côté, les enseignants pourront :

Consulter les informations sur les élèves : de manière centralisée et en temps réel, facilitant la gestion des élèves et l'ajustement des stratégies pédagogiques.

Envoyer des notifications et des messages : aux parents, et suivre l'engagement des parents dans le suivi de leur enfant.

Cette approche améliore la transparence et renforce l'implication des parents dans le processus éducatif, tout en allégeant la charge administrative des enseignants.

1.4.4 Intégration d'un module d'intelligence artificielle

Une des innovations majeures de cette solution CRM est l'intégration d'un module d'intelligence artificielle (IA). Ce module pourrait être utilisé pour analyser les données des élèves et identifier des tendances, telles que :

Le suivi des performances : des enfants et l'identification des domaines nécessitant une attention particulière.

La prédiction des besoins pédagogiques : En fonction des progrès individuels des élèves, permettant ainsi aux enseignants de personnaliser les cours et d'adapter leur méthode d'enseignement. L'IA pourrait aussi permettre de détecter des patterns dans les interactions entre enseignants et parents, offrant des recommandations pour améliorer l'engagement des familles. Enfin, l'IA pourrait être utilisée pour automatiser certaines tâches répétitives, comme la génération de rapports ou l'analyse des données administratives.

1.4.5 Sécurité et protection des données

La solution CRM mettra en place des protocoles de sécurité avancés pour protéger les données des utilisateurs. Voici quelques mesures envisagées :

Authentification sécurisée : Utilisation d'une authentification forte (par exemple, 2FA) pour garantir que seul le personnel autorisé puisse accéder à certaines informations sensibles.

Chiffrement des données : Toutes les données personnelles des utilisateurs (élèves, parents, enseignants) seront stockées et transmises de manière cryptée, assurant leur protection contre

toute tentative d'accès non autorisé.

Conformité aux réglementations : Le système sera conçu pour respecter les normes internationales de protection des données (comme le RGPD), en permettant aux utilisateurs de gérer leurs propres informations et en garantissant leur droit à la confidentialité.

1.4.6 Déploiement avec Kubernetes

Le déploiement de l'application se fera sur une infrastructure Kubernetes, garantissant une gestion optimale des ressources et une évolutivité automatique de l'application. Kubernetes permet une gestion simplifiée des conteneurs et facilite l'automatisation des déploiements, ce qui offre une haute disponibilité et une gestion dynamique des charges de travail.

Cela assure également une résilience élevée, car en cas de panne d'un service, Kubernetes permet de redémarrer automatiquement les conteneurs affectés sans perturber l'application dans son ensemble.

1.5 Méthodologie adoptée

Afin de réaliser le projet dans les délais établis par la convention de stage, il a fallu définir les étapes essentielles et estimer le temps à consacrer pour chacune. Pour cela, nous avons utilisé un processus de développement agile. L'objectif d'un processus de développement est de produire des logiciels de qualité qui répondent aux besoins de leurs utilisateurs dans des temps et des coûts prévisibles.

La méthode que nous avons donc choisie pour ce projet est la méthode agile SCRUM.

SCRUM est la méthode la plus connue des méthodologies agiles. En effet, elle a fait ses preuves dans des environnements de développement de projets qui connaissent des changements rapides et des exigences émergentes.

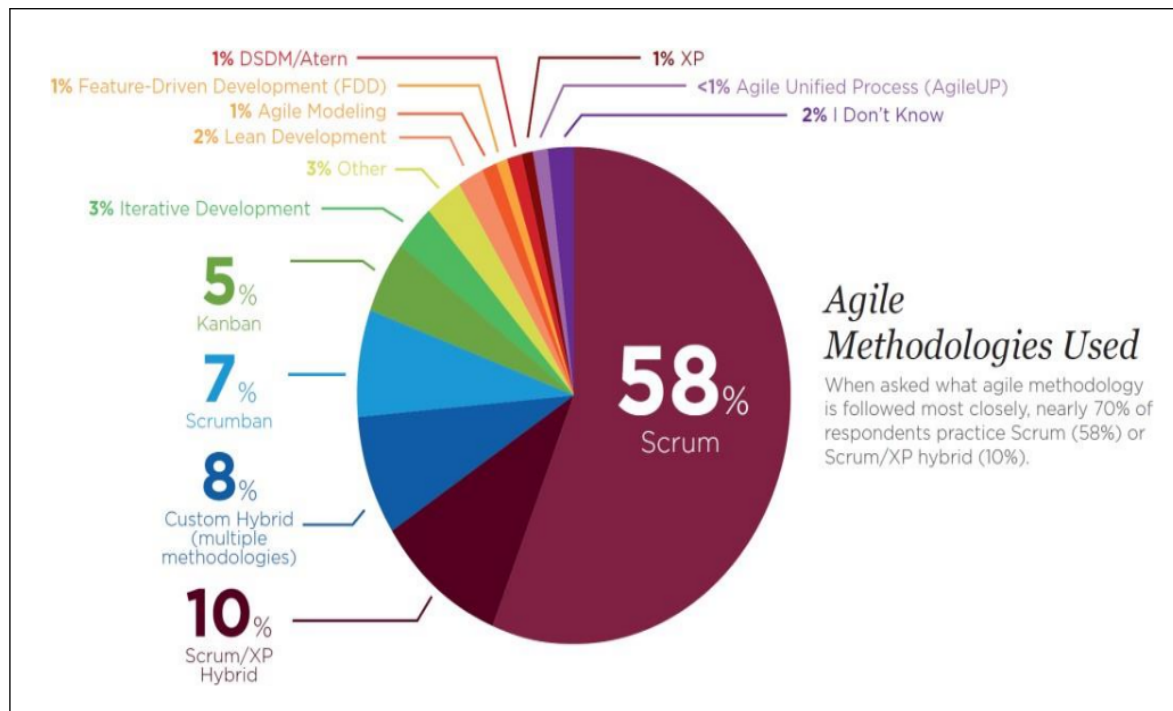


FIGURE 1.2 – Pourcentages des méthodes agiles utilisées [1]

1.5.1 Présentation de SCRUM

Scrum est une méthode agile de gestion de projet qui favorise le développement collaboratif, itératif et incrémental de logiciels. Elle repose sur des cycles de développement courts appelés sprints, généralement de deux à quatre semaines, durant lesquels des fonctionnalités spécifiques sont planifiées, développées, testées et révisées. Scrum met l'accent sur l'adaptabilité, la communication continue et le retour d'information régulier à travers des événements tels que la réunion quotidienne (daily scrum), la planification du sprint, la revue de sprint et la rétrospective. Cette approche améliore la transparence et permet à l'équipe de réagir rapidement aux évolutions des besoins des utilisateurs.

1.5.2 Formalisme adopté

Afin de mener efficacement notre cycle de développement, l'équipe aurait besoin des modélisations du produit logiciel. Nous avons choisi, pour cela, le formalisme UML (Unified Modeling Language), qui est un langage visuel constitué d'un ensemble de schémas, appelés des diagrammes, qui

donnent chacun une vision différente du projet à traiter. UML nous fournit donc des diagrammes pour représenter le logiciel à développer : son fonctionnement, sa mise en route, les actions susceptibles d'être effectuées par le logiciel, etc. Ainsi, UML définit neuf types de diagrammes répartis en deux catégories de vues : les vues statiques et les vues dynamiques. Les diagrammes UML sont :

Les diagrammes de cas d'utilisation, qui décrivent le comportement et les fonctions d'un système du point de vue de l'utilisateur ;

Les diagrammes de classes, qui décrivent la structure statique, les types et les relations des ensembles d'objets.

Tout au long du rapport, nous présenterons quelques diagrammes que nous avons jugés utiles et suffisants pour comprendre le projet, à savoir les diagrammes de cas d'utilisation.

1.6 CONCLUSION

Ce premier chapitre constitue une étape primordiale pour fixer les repères de notre projet. Après avoir présenté l'organisme d'accueil, nous avons déterminé le cadre du projet, les limites de l'existant qui ont engendré ce travail, ainsi que la méthodologie à emprunter lors de ce stage. Dans le prochain chapitre, nous allons aborder l'analyse et la spécification des besoins.

Analyse et spécification des besoins

Sommaire

2.1	INTRODUCTION	15
2.1.1	Analyse des besoins	15
2.2	Spécification des besoins	17
2.2.1	Diagramme de cas d'utilisation Global	17
2.3	Diagramme de classes	19
2.4	Planification de travail	20
2.4.1	Répartition des releases	20
2.4.2	Product backlog globale	22
2.5	Architecture de l'application	22
2.5.1	Architecture logique	22
2.5.2	Architecture physique	27
2.5.3	Architecture microservices	28
2.6	Environnement de travail	29
2.6.1	Environnements de développement matériel	29
2.6.2	Environnements de développement logiciel	30
2.7	CONCLUSION	30

2.1 INTRODUCTION

Le présent chapitre représente notre sprint de démarrage au cours duquel nous spécifierons les exigences des différents utilisateurs. Une étude des besoins fonctionnels et non fonctionnels est alors nécessaire. Ensuite, nous exposons notre backlog de produit ainsi que la planification de nos sprints. Enfin, nous expliquons l'architecture en microservices proposée ainsi que l'environnement de travail.

2.1.1 Analyse des besoins

Un acteur représente un rôle joué par une entité externe (utilisateur humain, dispositif matériel ou autre système) qui interagit directement avec le système étudié.

Un acteur peut consulter et/ou modifier directement l'état du système, en émettant et/ou en recevant des messages susceptibles d'être porteurs de données

Pour une bonne gestion du CRM, mon application est composée de plusieurs espaces et chaque espace est identifié par un ou plusieurs acteurs possédant des rôles et des fonctions bien spécifiques.

Dans Notre cas, les acteurs qui ont le droit d'accéder au module gestion de fournisseur est l'administrateur, le parent, l'enseignant et l'élève.

2.1.1.1 Analyse des besoins fonctionnels

Gestion des utilisateurs :

Authentification par 4 types d'utilisateurs : Administrateur, Enseignant, Élève, Parent.

Création, consultation, mise à jour et suppression (CRUD) des comptes utilisateurs.

Gestion des classes : Création et affichage des classes.

Association des enseignants à une ou plusieurs classes.

Association des élèves à une classe.

Affichage des élèves d'une classe (par enseignant ou administrateur).

Gestion des cours et contenus pédagogiques : Création de leçons associées à une classe.

Ajout de chapitres sous chaque leçon.

Téléversement de fichiers PDF pour chaque chapitre.

Affichage des cours (leçons/chapitres/PDF) côté élève.

Interface parent : Affichage des enfants liés à un parent.

Suivi des cours de leurs enfants.

Gestion des accès et sécurité : Contrôle des autorisations selon les rôles.

Sécurisation de l'authentification et des transferts de fichiers.

Tableau de bord dynamique (version future) : Visualisation dynamique des données (élèves, progression, fichiers, etc.).

Accès filtré selon le type d'utilisateur connecté.

Intégration de l'intelligence artificielle : Module d'IA pour le suivi pédagogique.

Génération de recommandations personnalisées pour les enseignants et les élèves.

Analyse des performances des élèves basée sur les interactions avec les contenus pédagogiques.

2.1.1.2 Analyse des besoins non fonctionnels

Les besoins non fonctionnels décrivent les propriétés que le système doit avoir. Dans ce qui suit nous détaillerons les besoins non fonctionnels que notre solution doit respecter :

- **Convivialité :** L'application doit être facile à utiliser. Les interfaces doivent respecter la charte graphique exigée par le client ;
- **Temps de réponse :** La nouvelle solution doit rendre une réponse dans un temps minimal ;
- **Ergonomie :** Les interfaces doivent être simples et faciles de manière à assurer une interaction avec l'utilisateur en informations claires et précises, et ceci pour réduire le temps d'apprentissage du logiciel ;
- **Évolutivité :** L'architecture doit être évolutive et extensible vu que les besoins du client varient ;
- **Maintenabilité :** La solution doit être maintenue, de manière cohérente et à moindre coût, en état de bon fonctionnement. C'est à dire, elle est simplement et rapidement réparée et ainsi à diminuer les temps et les coûts d'intervention

2.2 Spécification des besoins

2.2.1 Diagramme de cas d'utilisation Global

Les diagrammes de cas d'utilisation sont utilisés pour donner une vision globale du comportement fonctionnel d'un système logiciel. Ils permettent de recueillir, d'analyser et d'organiser les besoins, et de recenser les grandes fonctionnalités d'un système. Il s'agit donc de la première étape UML d'analyse d'un système. En effet, c'est une représentation de l'interaction des utilisateurs (acteurs) et le système (cas d'utilisation) sans se soucier de la manière dont le système va exécuter les fonctionnalités décrites. Le diagramme de cas d'utilisation global représenté par la figure 2.1, modélise les fonctionnalités globales offertes par notre application. Nous détaillerons dans les chapitres suivants les cas d'utilisation réalisés.

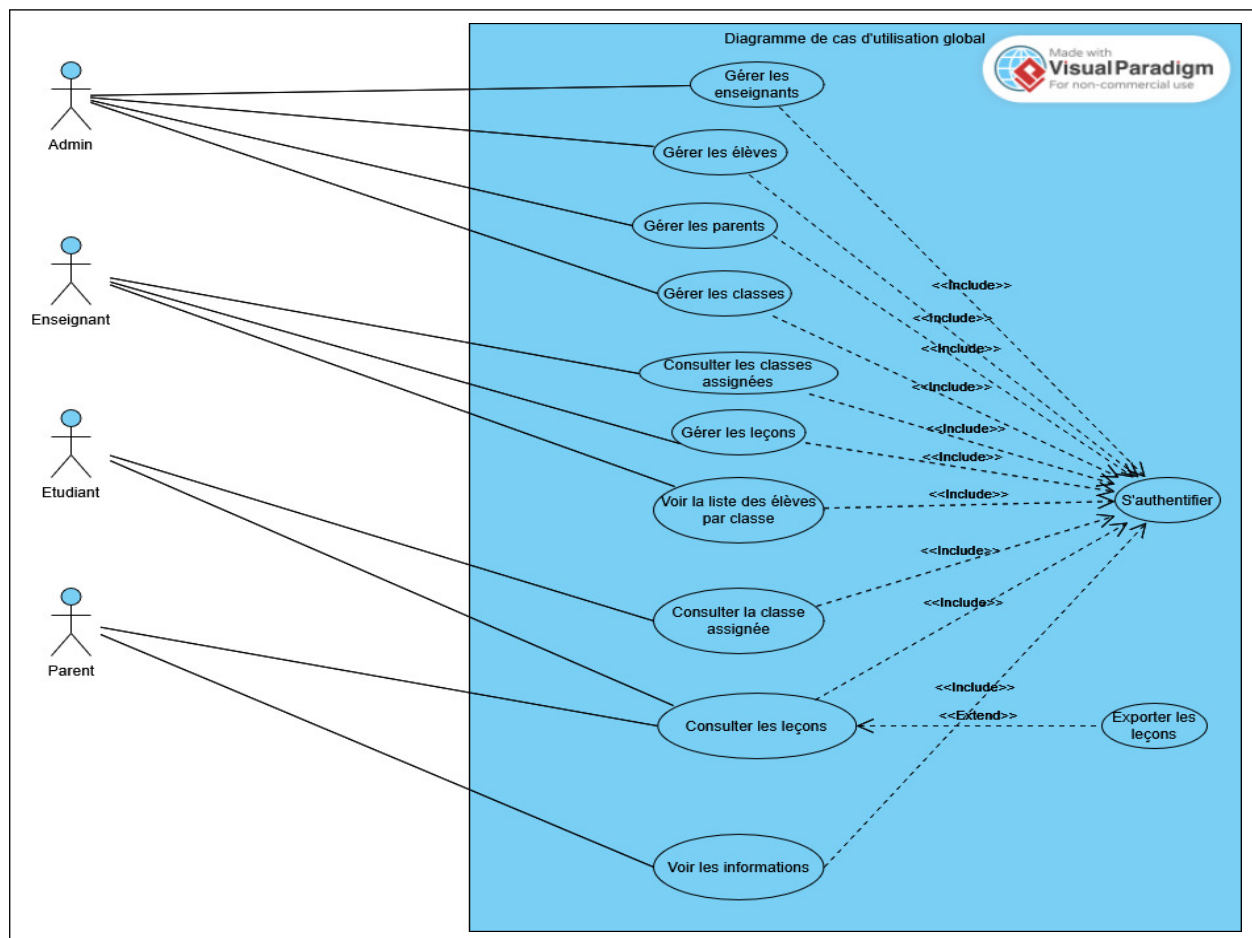


FIGURE 2.1 – Diagramme de cas d'utilisation global

Dans notre applications on a principalement 4 acteurs :

Administrateur :

- Gérer les enseignants (création, modification, suppression)
- Gérer les classes (organisation des classes)
- Gérer les leçons (planification du contenu pédagogique)

Enseignant :

- Consulter les classes assignées (voir les classes dont il est responsable)
- Consulter les leçons (accéder au contenu pédagogique)
- Voir la liste des élèves par classe (gestion des effectifs)

Élève :

- Consulter la classe assignée (connaître sa classe)
- Consulter les leçons (accéder au matériel pédagogique)

Parent

- Consulter les classes assignées (suivre la classe de son enfant)
- Voir les informations (accéder aux données scolaires)

2.3 Diagramme de classes

La figure 2.2 illustre le diagramme de classes du sprint 1 et les relations qui existent entre ses différentes classes. Nous devons préciser que chaque classe des diagrammes de classes des différents sprints contiennent à part les attributs, un constructeur par défaut, des constructeurs avec paramètres, des getters et des setters de tous les attributs

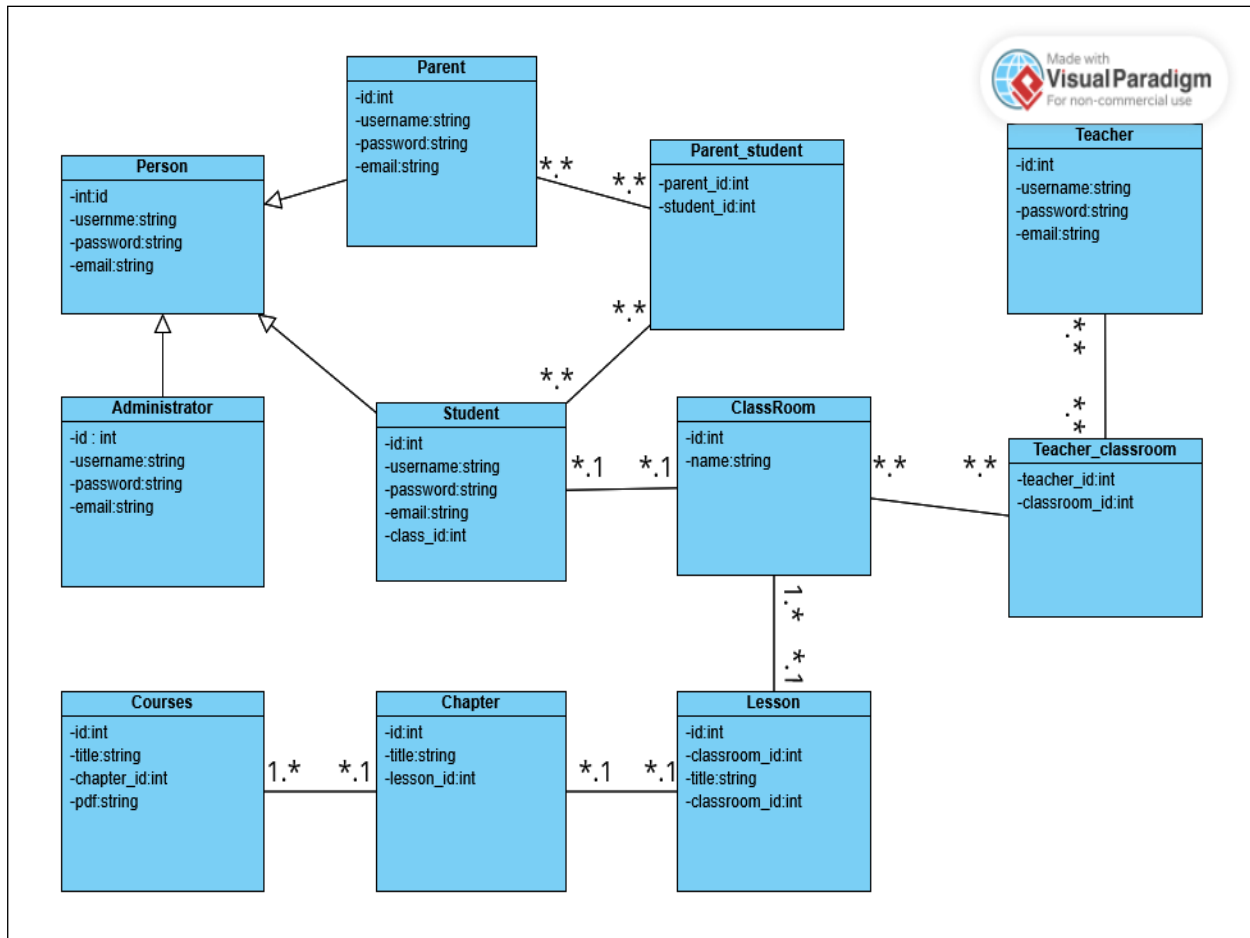


FIGURE 2.2 – Diagramme de classe global

Le diagramme de classes global de notre application représente l'ensemble des entités principales et les relations entre elles. Il est centré autour de la classe abstraite Utilisateur, héritée par quatre rôles distincts : Administrateur, Enseignant, Élève et Parent. Chaque enseignant peut être associé à une ou plusieurs classes, tandis que chaque élève appartient à une seule classe. Les parents, quant à eux, sont liés aux élèves pour assurer le suivi. Le système permet également la gestion pédagogique à travers les entités Leçon et Chapitre, associées aux classes

et aux enseignants. Chaque chapitre contient un fichier PDF téléversé comme support de cours. Ce modèle de données favorise une organisation claire des responsabilités, facilite l’affichage conditionnel des interfaces selon le rôle de l’utilisateur, et prépare le terrain à l’intégration de fonctionnalités avancées comme un module d’intelligence artificielle.

2.4 Planification de travail

2.4.1 Répartition des releases

Le tableau 2.1 représente la répartition des releases.

Sprint	Tâches principales
Sprint 1	— Authentification des 4 types d'utilisateurs (Administrateur, Enseignant, Élève, Parent) — Création et affichage des classes — Association enseignants classes et élèves classe
Sprint 2	— Gestion des cours : création de leçons, chapitres et importation de fichiers PDF — Affichage des cours côté élève — Mise en place de l'interface parent (consultation des enfants et de leurs cours)
Sprint 3	— Upload sécurisé des fichiers (validation, chemin, stockage) — Visualisation dynamique des données (ex : liste des cours, chapitres) — Implémentation des premières couches de sécurité
Sprint 4	— Phase de tests, corrections de bugs et déploiement — Lancement de l'architecture microservices (si la version MVP est stable) — Intégration d'un module d'IA pour le suivi pédagogique (analyse de progression des élèves, suggestions)

TABLE 2.1 – Répartition des releases par sprint avec intégration de l'IA

2.4.2 Product backlog globale

Le tableau 2.2 représente le product backlog globale .

ID	User Story	Priorité
1	En tant qu'administrateur, je peux me connecter à l'application	Haute
2	En tant qu'enseignant, je peux ajouter un cours pour une classe	Haute
3	En tant qu'élève, je peux consulter les cours associés à ma classe	Moyenne
4	En tant que parent, je peux voir les enfants qui me sont associés	Moyenne
5	En tant qu'administrateur, je peux gérer les documents administratifs	Basse
6	Ajouter un module de sécurité (ex. authentification sécurisée, droits d'accès)	Moyenne
7	Intégrer un module IA pour le suivi pédagogique des élèves	Faible
8	En tant qu'administrateur, je peux créer et gérer les classes	Haute
9	En tant qu'enseignant, je peux associer des chapitres et fichiers PDF aux leçons	Haute
10	En tant qu'utilisateur, je veux une interface fluide et sécurisée pour consulter mes informations	Moyenne

TABLE 2.2 – Product Backlog global de l'application

2.5 Architecture de l'application

2.5.1 Architecture logique

Couche d'affichage (Frontend Angular) : Cette couche gère l'interaction de l'utilisateur avec le système. Elle contient toutes les interfaces utilisateur pour les quatre profils – Administrateur, Enseignant, Étudiant, Parent. Elle consomme les API exposées par le backend pour afficher dynamiquement les données et permettre des actions (authentification, gestion des cours, consultation,

etc.).

Couche de logique métier (Django REST API) : Cette couche agit comme un médiateur entre l'interface utilisateur et la base de données. Elle contient la logique métier principale : vérification des droits, traitement des demandes, validation des données, etc. Chaque entité métier (Enseignant, Étudiant, Cours, etc.) a son propre contrôleur (ViewSet ou APIView) pour encapsuler la logique métier associée.

Couche de logique applicative (Django Models) : Cette couche représente les fonctionnalités principales du système. Elle se compose des classes Django définissant les entités métier telles que : Enseignant, Étudiant, Leçon, Chapitre, Cours, etc. Chaque classe est logiquement liée à d'autres classes par des relations (ForeignKey, ManyToMany...).

Couche d'accès aux données (Django ORM) : Gérée automatiquement par Django avec son ORM, cette couche permet l'interaction avec une base de données relationnelle sans avoir à écrire manuellement des instructions SQL. Elle garantit la persistance et le rec

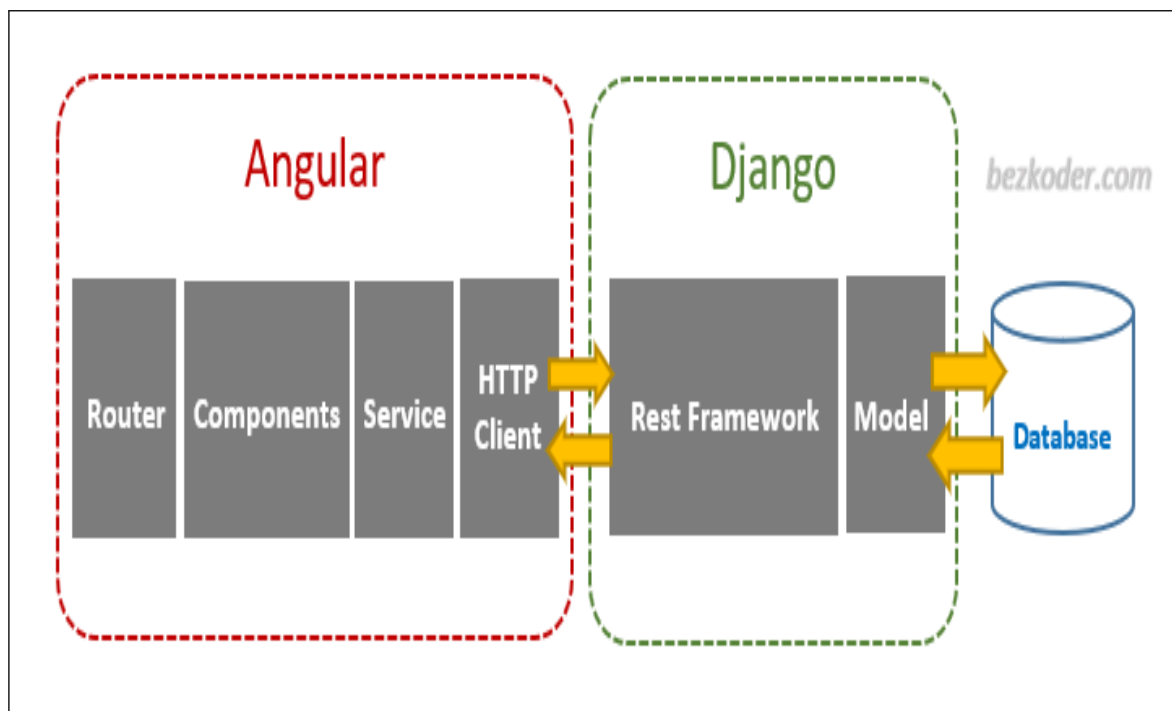


FIGURE 2.3 – Architecture logique de l'application [2]

2.5.1.1 Architecture logique frontEnd

Angular est construit sur une framework modulaire, basé sur des composants, des services et des modules, ce qui améliore la scalabilité, la réutilisabilité et le test unitaire. Ces derniers s'inscrivent dans le modèle MVC (Modèle Vue Contrôleur) dans sa forme modernisée et dédiée aux applications monopage.

Les principales briques logiques sont :

Module : ou NgModule qui permet de regrouper des composants afin de mieux structurer le code et faciliter la réutilisation et le partage. Le module racine, appelé AppModule, fournit le mécanisme d'amorçage qui lance l'application. **Component** : les composants sont les éléments fondamentaux d'une application Angular. Ils permettent d'afficher les données à l'écran, écouter les événements générés par l'utilisateur et lancer des actions basées sur ces événements. Un component est composé d'un contrôleur (.ts) d'une template (.html) et d'une feuille de style (.css). **Template** : c'est la combinaison du HTML avec les directives d'Angular, pouvant modifier les éléments HTML avant leur affichage. **Directives** : ils permettent de rendre une application dynamique. **Service** : un service est une classe qui contiendra une logique que l'on va pouvoir utiliser un peu partout dans notre application. L'injecteur de dépendances va nous permettre ainsi d'injecter nos services seulement aux endroits souhaités et éviter de répéter du code logique dans chaque composant. **Routing** : le router est un module permettant de définir un chemin de navigation entre les différentes vues. **HTTPClient** : HttpClient est utilisé par Angular pour interagir avec le backend (par exemple, Django REST API).

Les appels sont couramment émis des services.

Formulaires : Angular offre deux approches de gestion des formulaires : template-driven et reactive forms.

Gestion d'État : Pour un projet complexe, Angular peut mettre en œuvre une structure de gestion d'état comme NgRx (Redux-like).

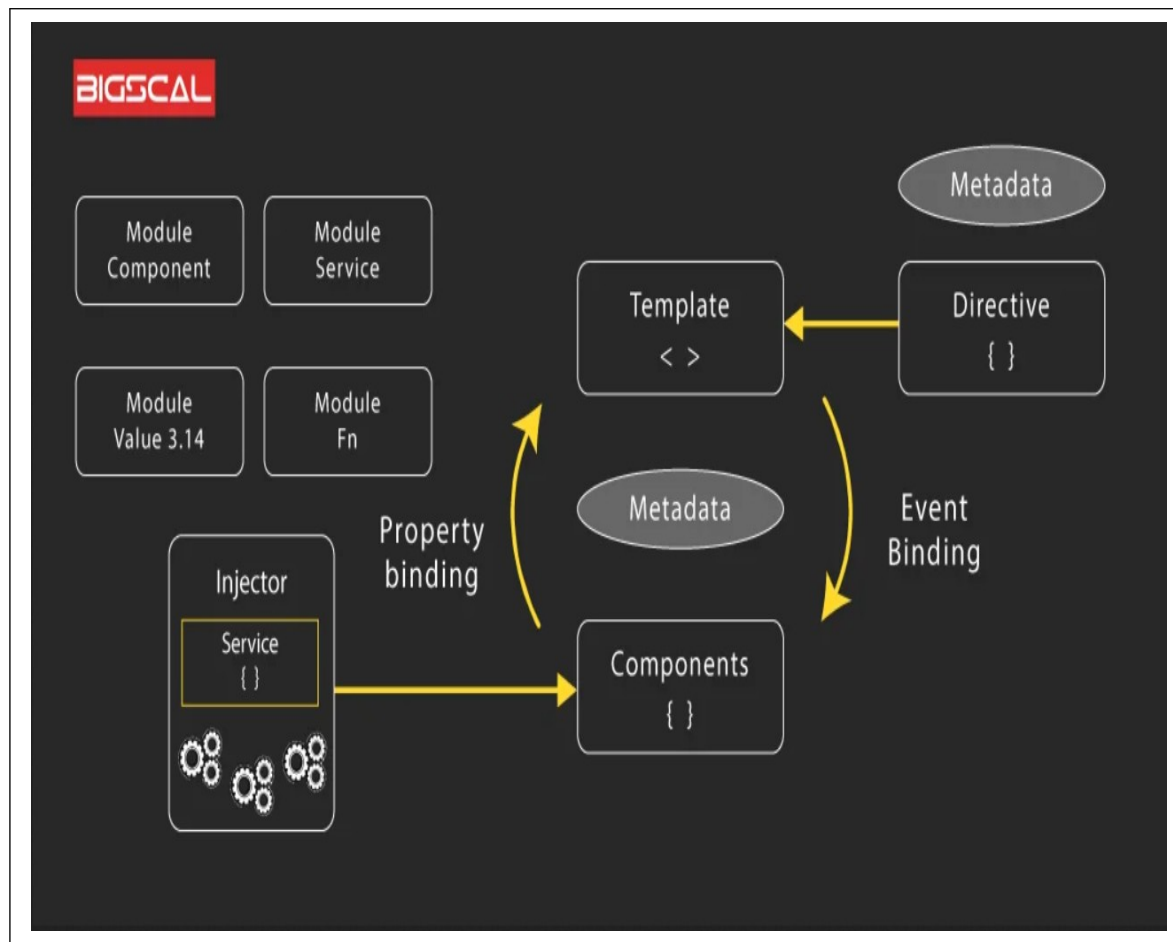


FIGURE 2.4 – Architecture logique Front-End (Angular) [3]

2.5.1.2 Architecture logique de Django

Création et déploiement d'en-tête Le backend de l'application est développé en utilisant le framework Django ainsi que son module : Django REST Framework (DRF). Il adopte les principes de l'API RESTful, qui offre une distinction claire entre la logique métier et l'interface utilisateur. La couche Modèle (Model) est implémentée à l'aide de structures de données qui sont fournies sous forme de classes via l'ORM Django. Les données de la couche Modèle sont gérées dans la couche Vue (View) à travers des ViewSets et des API Views qui exécutent des requêtes HTTP, manipulent le modèle et renvoient les données au format JSON. Les templates Django sont omis dans cette architecture car toute la logique de présentation est gérée par l'interface avant développée en Angular. Cette architecture logique améliore de manière frappante la flexibilité, la scalabilité : surtout pour les microservices, et rationalise la communication entre l'interface avant et le backend grâce aux appels API. **Modèles (Models) :** Les modèles sont

définis à l'aide de l'ORM Django intégré. Une classe Python correspond à une table de base de données. Ces modèles décrivent les entités clés de l'application, comme Student, Teacher, Lesson, etc., et leurs relations (ForeignKey, ManyToMany).

Sérialiseurs (Serializers) : DRF serializers allow for the serialization of Python objects (from the models) to JSON data that can be consumed by the frontend. They also offer validation for incoming data (POST/PUT).

Vues (ViewSets et APIViews) : ViewSets allow for the fast creation of CRUD operations via automatically generated routes. Custom APIView is utilized for special actions, like login (AdminLoginView) or getting children for a parent (ParentChildrenView).

Routeurs (Routers) : Le module routers de DRF permet d'acheminer automatiquement les requêtes HTTP vers les bonnes vues, facilitant la structure et la lisibilité du code.

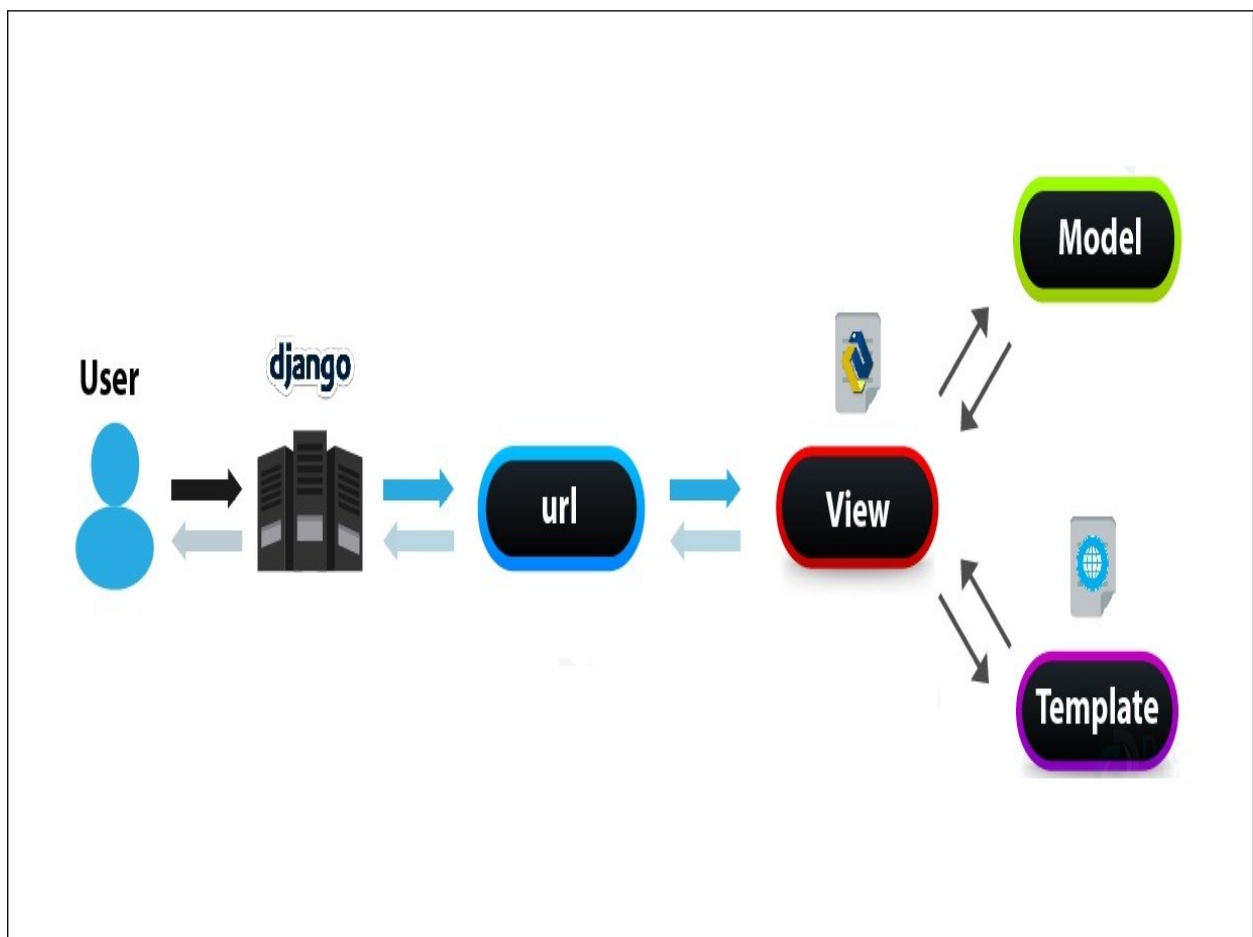


FIGURE 2.5 – Architecture logique du backend (Django) [4]

2.5.2 Architecture physique

L'architecture physique, également nommée architecture technique, décrit l'ensemble des composants matériels supportant l'application.

La figure 2.6 ci-dessous décrit l'architecture physique de notre application. Elle est composée essentiellement de : **Client (Frontend)** : Les utilisateurs accèdent à l'application via un navigateur.

L'interface est développée en Angular et servie via un serveur web (Nginx, par exemple).

Serveur Web (API) : Le backend est développé avec Django et expose une API REST (Django REST Framework). Il est hébergé sur un serveur applicatif (ou conteneur Docker).

Base de données : Toutes les données (utilisateurs, classes, cours, etc.) sont stockées dans une base relationnelle telle que PostgreSQL ou MySQL, hébergée sur un serveur séparé ou un service managé.

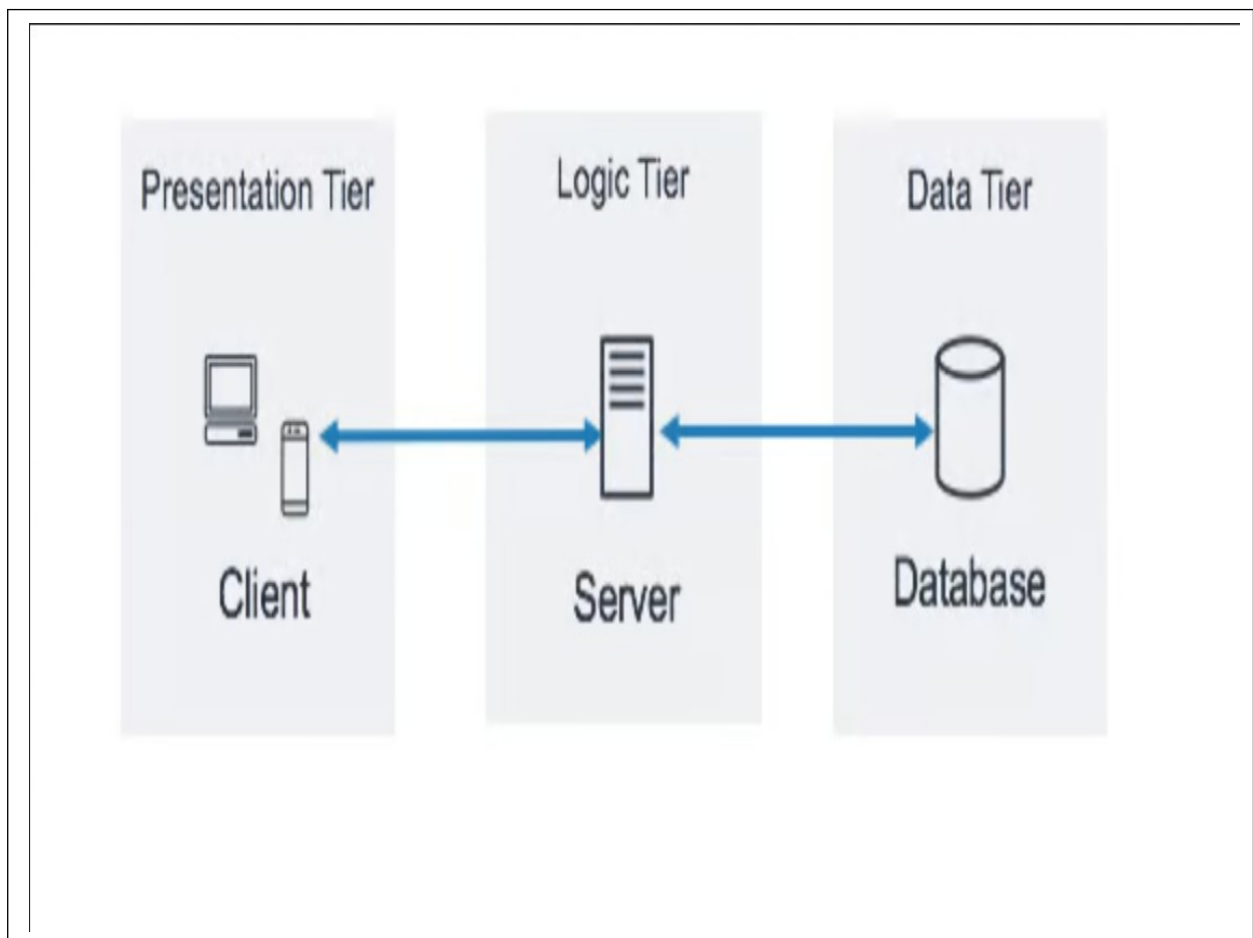


FIGURE 2.6 – Architecture physique globale [5]

2.5.3 Architecture microservices

Les microservices désignent à la fois une architecture et une approche de développement logiciel.

Cette approche consiste à décomposer les applications en éléments les plus simples, indépendants les uns des autres. Contrairement à une approche monolithique classique, selon laquelle tous les composants forment une entité indissociable, les microservices fonctionnent en synergie pour accomplir les mêmes tâches, tout en étant séparés. Chacun de ces composants ou processus est un microservice[8]. Ces processus peuvent communiquer ensemble afin de fournir les fonctionnalités nécessaires.

Les microservices sont, dans la plupart du temps, implémentés et pilotés par des équipes de petite taille avec suffisamment d'autonomie. Chacune peut changer l'implémentation de chaque microservice, ajouter ou supprimer des fonctionnalités de ce service avec un impact minimal sur les autres microservices. Ce style d'architecture présente plusieurs avantages comme l'hétérogénéité technologique, la résistance contre l'échec, la scalabilité sur mesure, la facilité de déploiement, l'alignement organisationnel, la réutilisabilité.

La figure 2.7 met l'accent sur la différence entre l'architecture monolithique et l'architecture microservices.

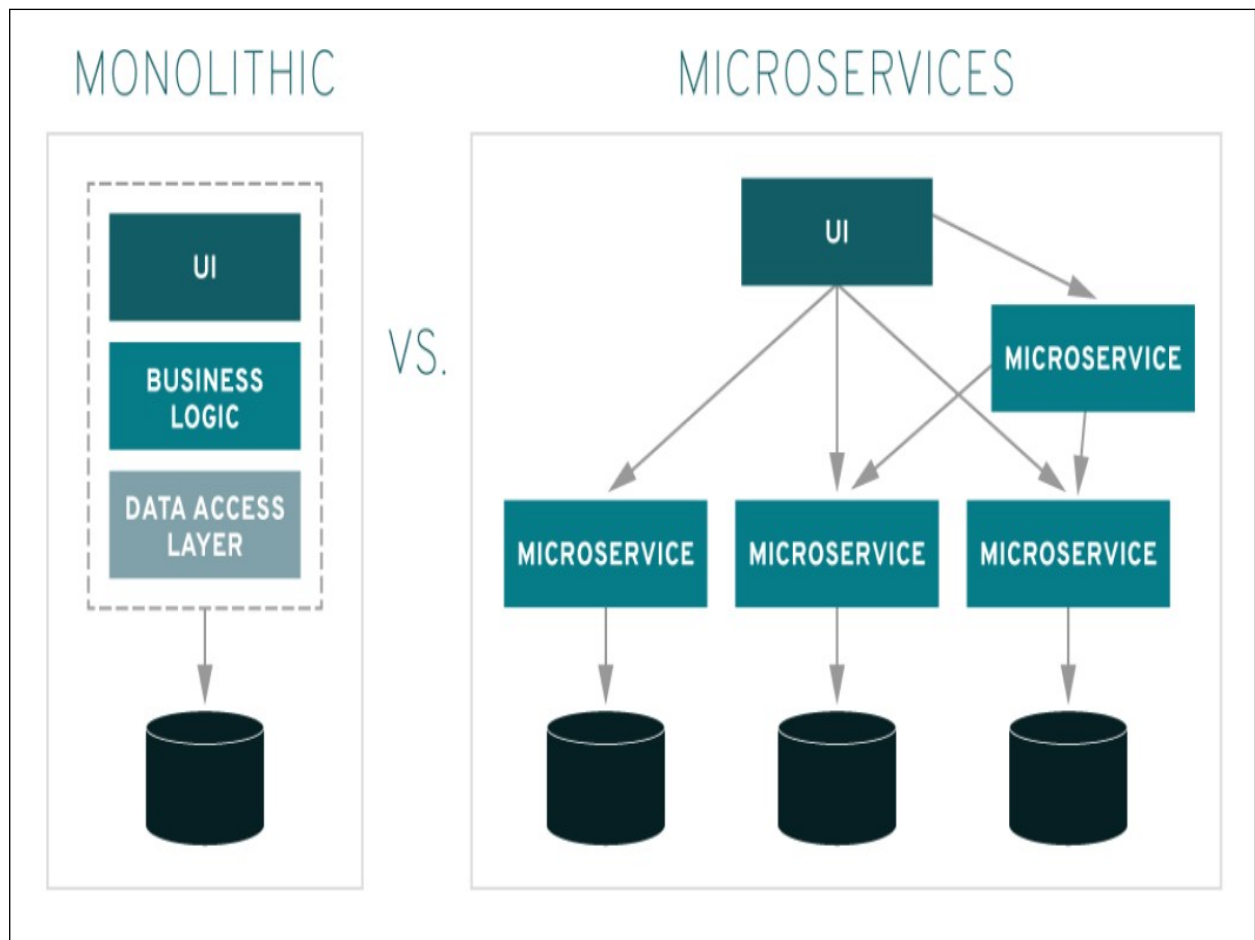


FIGURE 2.7 – Architecture monolithique versus architecture microservices [6]

2.6 Environnement de travail

Tout au long de la réalisation de notre projet, nous avons utilisé des matériels et des logiciels bien particuliers que nous présenterons dans ce qui suit.

2.6.1 Environnements de développement matériel

Pour mener à bien la réalisation, nous avons utilisé comme environnement matériel, un poste de travail ayant les caractéristiques suivantes :

Système d'exploitation	Windows 11
Processeur	Intel Core i7, 13 ^e génération
RAM	16 Go
Disque dur	500 Go

TABLE 2.3 – Caractéristiques de la machine utilisée

2.6.2 Environnements de développement logiciel

Les outils logiciels qui nous ont servi à réaliser notre application sont les suivants **Visual Studio Cde** : c'est un éditeur de cde pen-surce, gratuit et multi-platefrme (Windws, Mac et Linux), dévelppé par Miersft. Principalement cnçu pur le développement d'application avec JavaScript, TypeScript et Node.js, l'éditeur peut s'adapter à d'autres types de langages grâce à un système d'extension bien fourni.

Postman : c'est un outil qui permet de construire et de tester des API REST.

MySQL Workbench : environnement visuel intégré pour la gestion des bases de données MySQL. Il permet la modélisation de données, l'exécution de requêtes SQL, la conception de schémas, et la gestion des utilisateurs et permissions.

Angular CLI : outil en ligne de commande utilisé pour créer, développer, tester et déployer des applications Angular. Il automatise la configuration initiale et fournit des commandes puissantes pour la gestion du projet.

2.7 CONCLUSION

Tout au long de ce chapitre, nous avons spécifié, tout d'abord, nos besoins et nous avons planifié nos sprints ce qui nous a aidé à avoir une vision plus claire, plus profonde et plus organisée sur notre projet. Ensuite, nous avons présenté l'architecture microservices qui composent notre application et nous avons détaillé l'architecture de la partie front-end, l'architecture de la partie back-end et l'architecture physique globale. Puis, nous avons décrit les environnements

matériels et logiciels sur lesquels nous avons travaillé et les Frameworks et technologies que nous avons utilisées. Dans le chapitre suivant, nous abordons notre premier sprint

Release

Sommaire

3.1	INTRODUCTION	33
3.2	Sprint 1 : Authentification et gestion des classes	33
3.2.1	Objectifs du sprint 1	33
3.2.2	Analyse du sprint 1	34
3.2.3	Diagramme de cas d'utilisation	35
3.2.4	Description textuelle des cas d'utilisation	36
3.2.5	Diagramme de classes	39
3.2.6	Réalisation	40

3.1 INTRODUCTION

Après avoir analysé et spécifié les besoins globaux de notre client, nous détaillerons les différentes étapes réalisées durant ce premier sprint. We have started to implement the fundamental functions of the system, especially the authentication of four types of users (Administrateur, Enseignant, Élève, Parent). Ensuite, nous avons conçu and implémenté la gestion des classes, permettant leur création et affichage. Enfin, nous avons développé les mécanismes d'association entre enseignants et classes, ainsi qu'entre élèves et leur classe respective. Ce chapitre présente également le backlog de ce sprint, une analyse des fonctionnalités développées, ainsi que les interfaces utilisateur associées.

3.2 Sprint 1 : Authentification et gestion des classes

3.2.1 Objectifs du sprint 1

L'objectif principal du premier sprint est de mettre en place l'authentification des différents types d'utilisateurs (Administrateur, Enseignant, Élève, Parent), ainsi que la création et la gestion des classes.

Il inclut également l'association des enseignants aux classes et des élèves à leurs classes respectives, constituant ainsi la base de l'organisation pédagogique de l'application.

ID	User Story	Priorité
1	En tant qu'administrateur, je peux m'authentifier pour accéder aux fonctionnalités de gestion.	Haute
2	En tant qu'enseignant, je peux me connecter pour gérer mes cours et ma classe.	Haute
3	En tant qu'élève, je peux me connecter pour consulter mes cours.	Haute
4	En tant que parent, je peux me connecter pour suivre les activités de mon enfant.	Moyenne
5	En tant qu'administrateur, je peux créer une classe et l'afficher dans l'interface.	Haute
6	En tant qu'administrateur, je peux associer un enseignant à une ou plusieurs classes.	Haute
7	En tant qu'administrateur, je peux associer un élève à une classe.	Haute
8	En tant qu'enseignant, je peux consulter la liste des élèves de ma classe.	Moyenne

TABLE 3.1 – Backlog du Sprint 1

3.2.2 Analyse du sprint 1

Dans cette partie, nous détaillerons les cas d'utilisation de la première itération, à savoir la gestion des fournisseurs et des sous-traitants. Nous nous référons aux diagrammes d'UML : les diagrammes de cas d'utilisation et les diagrammes de séquence système.

3.2.3 Diagramme de cas d'utilisation

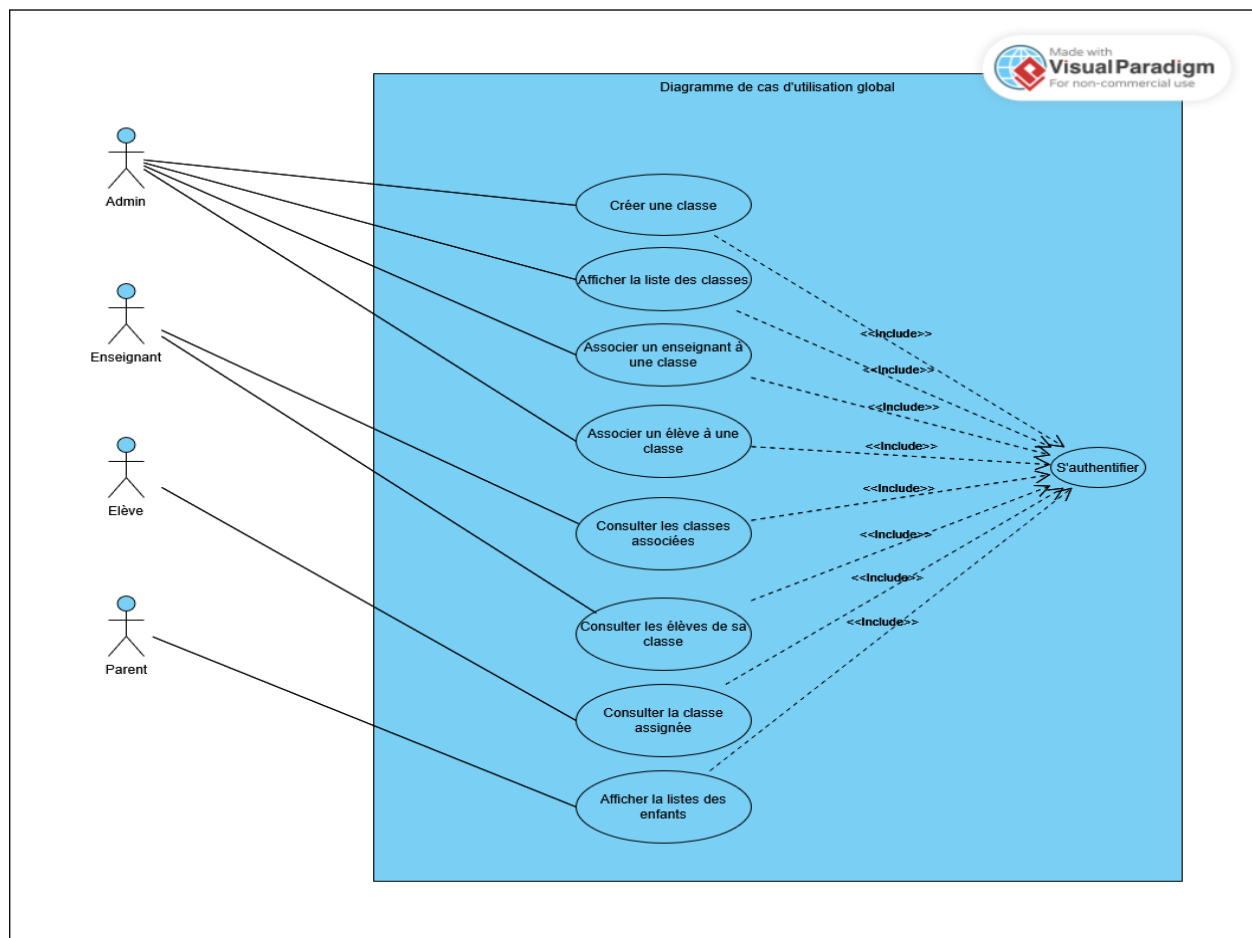


FIGURE 3.1 – Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les fonctionnalités principales du système pour le Sprint 1, incluant les acteurs Admin, Enseignant, Élève et Parent. Les principales actions couvertes sont la création et gestion des classes (création, affichage, association d'enseignants et d'élèves), ainsi que diverses fonctionnalités de consultation permettant aux enseignants de voir leurs classes et élèves, aux élèves de consulter leur classe assignée, et aux parents d'afficher la liste de leurs enfants. Ce diagramme a été réalisé avec Visual Paradigm pour un usage non-commercial.

3.2.4 Description textuelle des cas d'utilisation

La description textuelle d'un cas d'utilisation permet de clarifier le déroulement de la fonctionnalité décrite et de préciser la chronologie des actions réalisées par l'acteur.

3.2.4.1 Description textuelle du cas d'utilisation « S'authentifier »

Acteur	Utilisateur
Objectif	S'authentifier
Scénario Principal	1. L'utilisateur saisit son email et son mot de passe. 2. Le système vérifie l'existence d'un utilisateur avec l'email et le mot de passe saisis. 3. Le système affecte les droits d'accès à l'utilisateur. 4. Le système génère un « token » sous format JWT. 5. Le système sauvegarde ces informations dans les « cookies ». 6. Le système redirige l'utilisateur vers sa page selon ses droits d'accès.
Scénario alternatif	A1 : Utilisateur inexistant ou mot de passe incorrect. 1. Le système indique à l'utilisateur que les informations saisies sont erronées ou non valables. 2. Le scénario principal reprend à la première étape.

TABLE 3.2 – Description textuelle du cas d'utilisation « S'authentifier »

3.2.4.2 Description textuelle du cas d'utilisation «Créer une classe»

Acteur	Administrateur
Objectif	Créer une classe dans le système
Scénario Principal	1. L'administrateur accède à l'interface de gestion des classes. 2. Il clique sur "Créer une classe". 3. Il saisit les informations nécessaires : nom de la classe, niveau, etc. 4. Le système vérifie la validité des informations. 5. Le système enregistre la classe dans la base de données. 6. Un message de confirmation s'affiche.
Scénario Alternatif	A1 : Informations manquantes ou invalides 1. Le système signale les erreurs à l'utilisateur. 2. L'administrateur corrige les champs puis soumet à nouveau.

TABLE 3.3 – Description textuelle du cas d'utilisation « Créer une classe »

3.2.4.3 Description textuelle du cas d'utilisation «Associer un enseignant à une classe»

Acteur	Administrateur
Objectif	Associer un enseignant à une ou plusieurs classes
Scénario Principal	1. L'administrateur accède à l'interface de gestion des affectations. 2. Il sélectionne un enseignant dans la liste. 3. Il choisit une ou plusieurs classes à associer. 4. Le système vérifie la validité de l'affectation. 5. Le système enregistre l'association. 6. Un message de confirmation s'affiche.
Scénario Alternatif	A1 : Enseignant ou classe introuvable 1. Le système affiche un message d'erreur. 2. L'administrateur relance l'opération avec des données valides.

TABLE 3.4 – Description textuelle du cas d'utilisation « Associer un enseignant à une classe »

3.2.5 Diagramme de classes

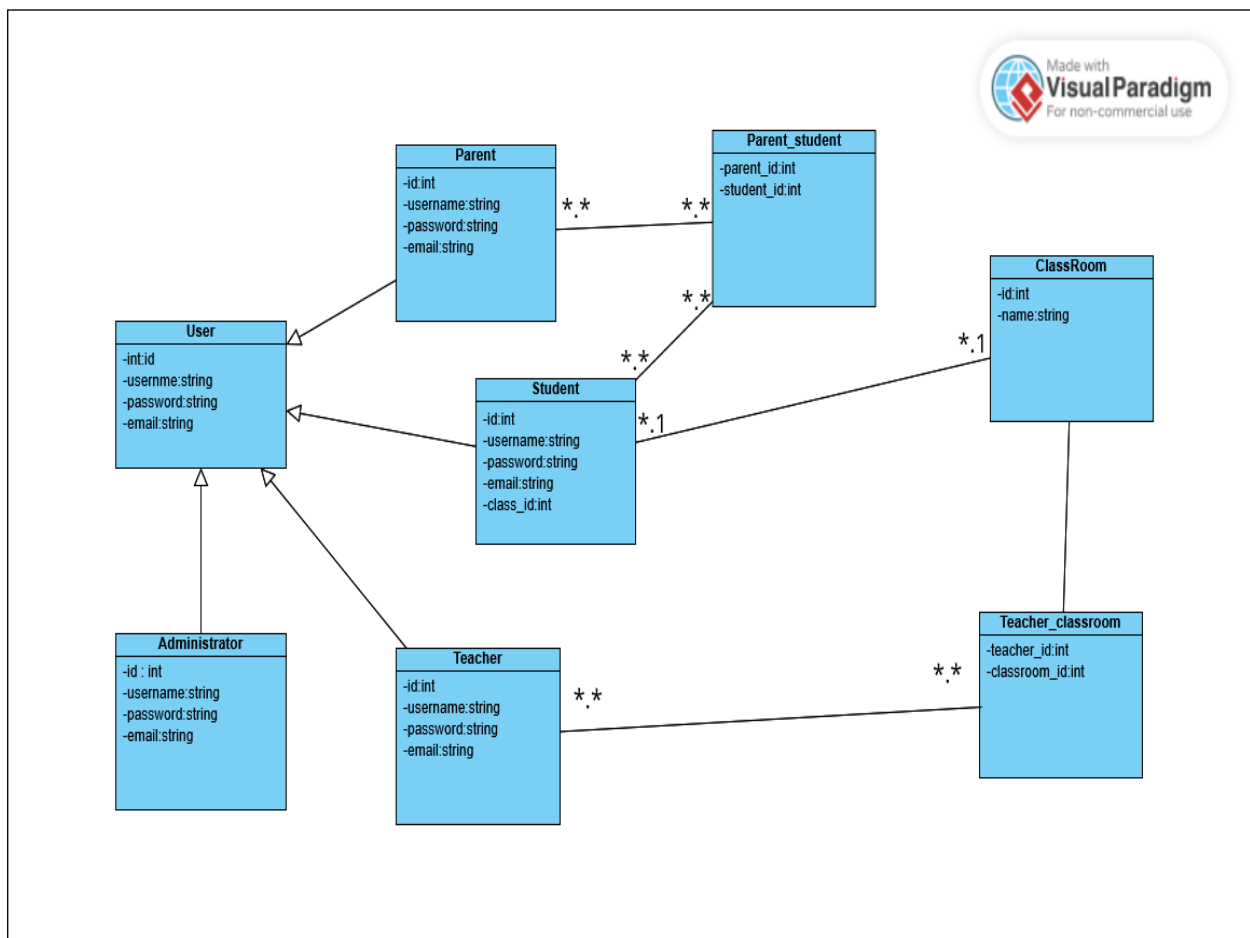


FIGURE 3.2 – Diagramme de classes

La figure 3.2 illustre le diagramme de classes du sprint 1 et les relations qui existent entre ses différentes classes :

Utilisateur : classe de base représentant un utilisateur générique avec des attributs communs (nom, email, mot de passe, etc.).

Administrateur, Enseignant, Élève, Parent : classes héritées de Utilisateur, chacune avec des rôles spécifiques.

Classe : représente une classe scolaire, liée à plusieurs élèves et enseignants.

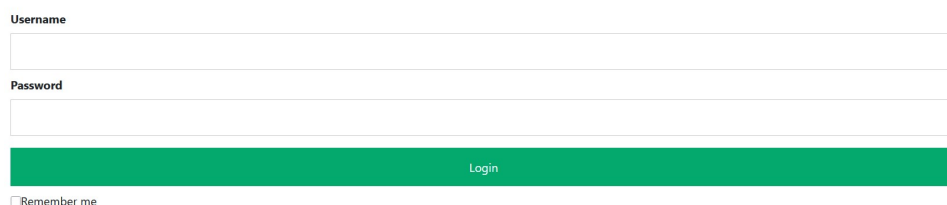
LienParentEnfant : gère la relation entre un parent et son ou ses enfants.

3.2.6 Réalisation

3.2.6.1 Technologies utilisées

Domaine	Technologie	Rôle
Frontend	Angular	Développement de l'interface utilisateur (authentification, affichage des classes, etc.)
Backend	Django	Gestion de la logique métier, API REST pour l'authentification et la gestion des classes
API	Django REST Framework (DRF)	Création des endpoints pour les opérations CRUD et l'authentification
Base de données	MySQL (via Workbench)	Stockage des données utilisateurs, classes, rôles
Test API	Postman	Test des routes d'authentification et de création de classes
IDE	Visual Studio Code	Édition du code source frontend et backend

TABLE 3.5 – Technologies utilisées lors du Sprint 1



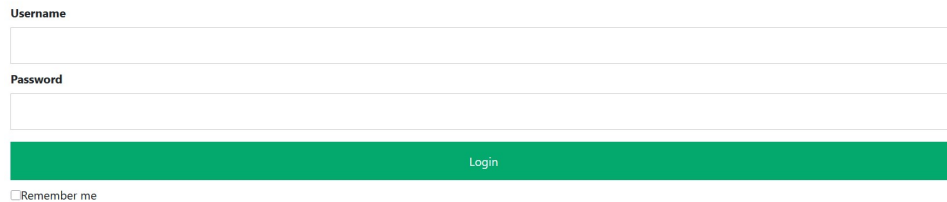
The image shows a login form with the following elements:

- A label "Username" above a text input field.
- A label "Password" above a text input field.
- A green button labeled "Login" below the password field.
- A checkbox labeled "Remember me" at the bottom left.

FIGURE 3.3 – Interface d'authentification

Après avoir effectué l'authentification avec succès, l'utilisateur accède à son espace personnel selon son rôle. Si l'utilisateur est un administrateur, il peut choisir l'option « Gestion des classes » dans le menu. Il sera alors redirigé vers l'interface dédiée à l'administration des classes. Dans le cas où l'administrateur souhaite créer une nouvelle classe, un formulaire de création lui sera

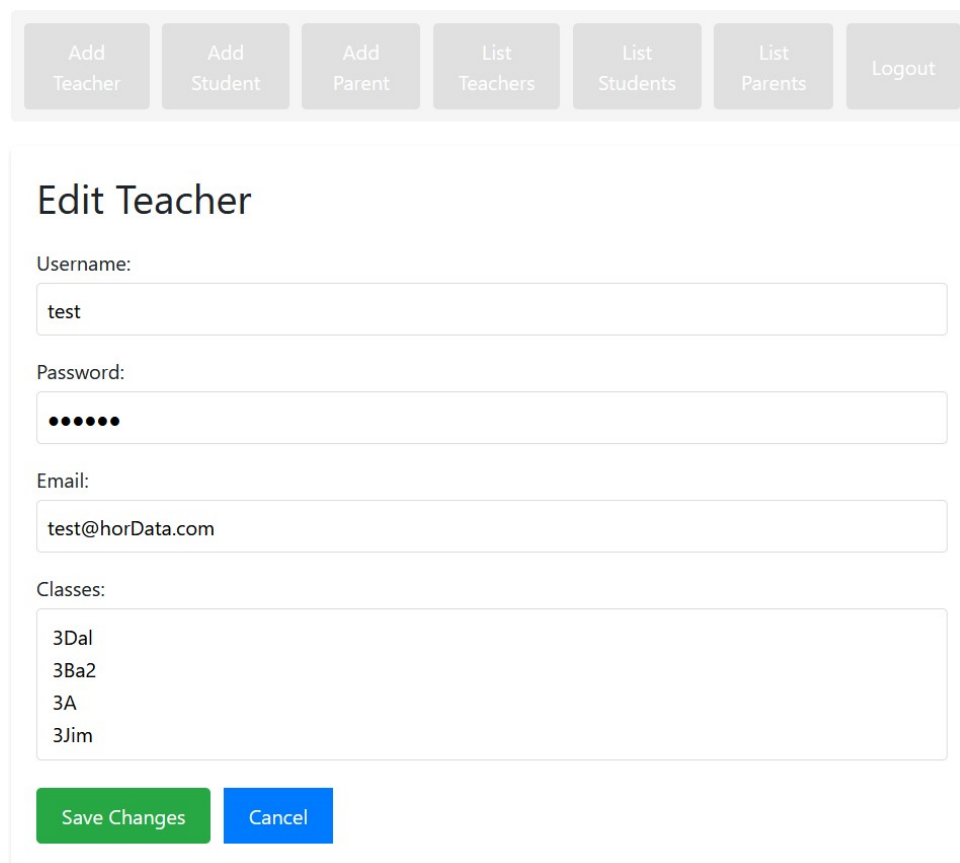
présenté pour saisir les informations nécessaires. L'utilisateur peut naviguer entre les différentes fonctionnalités de gestion via les icônes ou les onglets disponibles, comme le montrent les figures suivantes.



A login form with two input fields: 'Username' and 'Password'. Below the password field is a green 'Login' button. At the bottom left, there is a checkbox labeled 'Remember me'.

FIGURE 3.4 – Interface d’authentification

Après avoir effectué l’authentification avec succès, l’utilisateur accède à son espace personnel selon son rôle. Dans le cas de l’administrateur, ce dernier peut gérer les affectations des enseignants via la page de modification d’un enseignant. En accédant à cette interface, l’administrateur peut sélectionner une ou plusieurs classes à associer à l’enseignant concerné. Le formulaire de modification permet une navigation claire entre les informations personnelles et les affectations pédagogiques, comme le montrent les figures suivantes.



The 'Edit Teacher' form is displayed below a navigation bar. The navigation bar contains buttons for 'Add Teacher', 'Add Student', 'Add Parent', 'List Teachers', 'List Students', 'List Parents', and 'Logout'. The 'Edit Teacher' form includes fields for 'Username' (containing 'test'), 'Password' (masked with dots), and 'Email' (containing 'test@horData.com'). Below these is a 'Classes' section with a list of checkboxes and labels: '3Dal', '3Ba2', '3A', and '3Jim'. At the bottom are 'Save Changes' and 'Cancel' buttons.

FIGURE 3.5 – modification d’un enseignant

diagram class na9es parent Une fois connecté, l'enseignant accède à un tableau de bord personnalisé lui permettant de consulter les informations relatives à sa ou ses classes. Parmi les fonctionnalités disponibles, il peut visualiser la liste des élèves associés à chacune de ses classes, ce qui facilite le suivi pédagogique au quotidien.

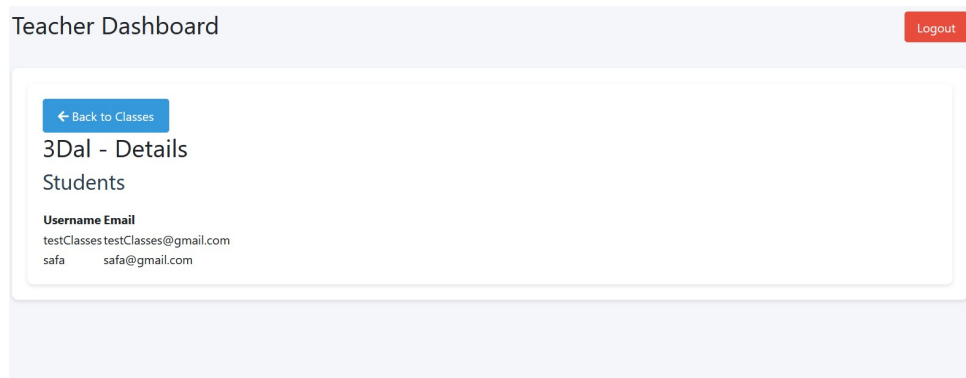


FIGURE 3.6 – Interface enseignant : liste des élèves de sa classe



CONCLUSION GÉNÉRALE

Dans un monde éducatif de plus en plus digitalisé, disposer d'une solution logicielle efficace, évolutive et adaptée aux besoins des différents acteurs devient une nécessité. La mise en place d'un système d'information éducatif permet non seulement de centraliser les données, mais aussi d'optimiser la communication et le suivi pédagogique.

Le projet que nous avons mené s'inscrit dans cette dynamique d'innovation. Il a pour but de concevoir et développer une application web éducative offrant un espace personnalisé pour chaque type d'utilisateur : administrateur, enseignant, élève et parent. Cette solution, construite à partir d'une architecture modulaire et orientée microservices, s'appuie sur les technologies Angular pour l'interface utilisateur et Django REST Framework pour la couche back-end.

Afin de concrétiser cette solution, nous avons suivi une approche agile basée sur le framework Scrum. Le projet a été découpé en plusieurs sprints, chacun visant à réaliser un ensemble de fonctionnalités spécifiques, comme l'authentification, la gestion des classes ou encore l'association des enseignants et des élèves.

Au cours de ce travail, nous avons mis en œuvre des principes d'architecture logicielle modernes et assuré la cohérence entre les modules tout en garantissant la maintenabilité et l'évolutivité du système. Plusieurs défis techniques se sont posés, notamment en matière de gestion des rôles, de sécurisation des échanges, de structuration des API ou de gestion des dépendances entre les composants.

Ce rapport a présenté en détail l'ensemble des étapes suivies, depuis la définition des besoins jusqu'à la réalisation technique. Nous avons abordé l'analyse fonctionnelle, la modélisation UML, la conception logicielle, le développement et la mise en œuvre des différentes interfaces utilisateurs.

Pour aller plus loin, nous envisageons de faire évoluer l'application en intégrant des modules d'intelligence artificielle, en automatisant davantage les tests, et en déployant l'infrastructure sur un environnement cloud (via Kubernetes). Cela permettrait d'améliorer encore plus la scalabilité, la robustesse et l'accessibilité du système, tout en répondant aux besoins futurs des utilisateurs.



BIBLIOGRAPHIE

- [1] **Emulent**. digital marketing strategists and creatives based in Raleigh [**en ligne**].
<https://emulent.com/blog/agile-seo/>
- [2] **bezkoder** [**en ligne**]. Disponible sur :
<https://www.bezkoder.com/django-angular-11-crud-rest-framework/>
- [3] **bigscal**. Architecture logique Front-End [**en ligne**]. Disponible sur :
<https://www.bigscal.com/blogs/frontend/angular-architecture-concepts-and-patterns/>
- [4] **data-flair.training**. Disponible sur :
<https://data-flair.training/blogs/django-architecture/>
- [5] **datacamp**. Disponible sur :
<https://www.datacamp.com/fr/blog/cloud-computing-projects>
- [6] **redhat**. Red Hat® Enterprise Linux®.
<https://linchpinseo.com/the-agile-method/>